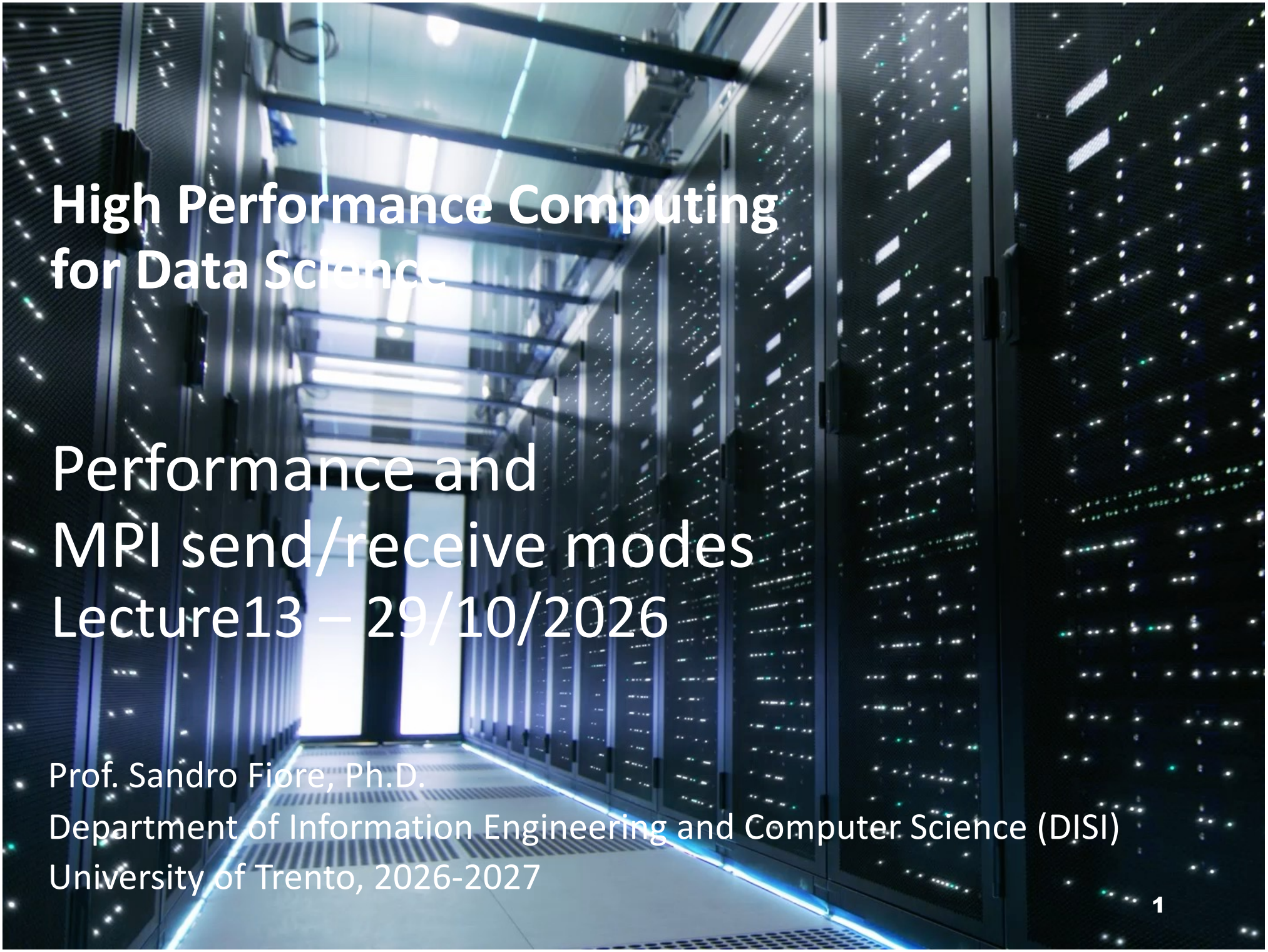


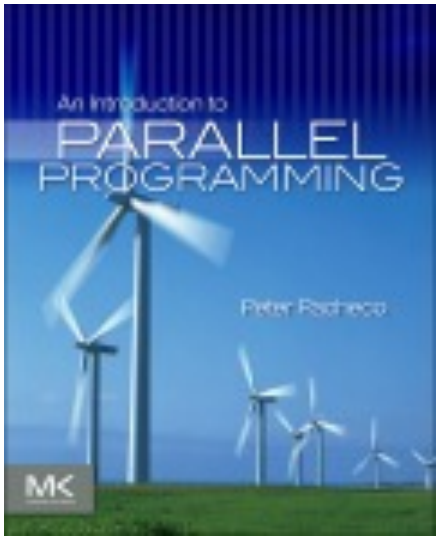
# High Performance Computing for Data Science

# Performance and MPI send/receive modes
# Lecture13 – 29/10/2026

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

# This week's outline and material

- Brief recap from the past week
- Performance
- MPI Send and Receive (comprehensive view)




## Today's material

Book
*An Introduction to Parallel Programming by Peter Pacheco*

Chapter 2 & 3
*Parallel Hardware and Parallel Software*
*Distributed Memory Programming with MPI*

# High Performance Computing for Data Science

Performance and MPI send/receive modes
Lecture 13 - 29/10/2026

## Recap

Prof. Sandro Fiore, Ph.D.
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

# Derived datatypes

- Used to represent any collection of data items in memory by storing both the **types** of the items and **their relative locations** in memory.
- The idea is that if a function that sends data knows this information about a collection of data items, it can **collect the items from memory before they are sent**.
- Similarly, a function that receives data can distribute the items into their correct destinations in memory when they’re received.


Copyright © 2010, Elsevier Inc. All rights Reserved

# Derived datatypes

- *Formally, consists of a sequence of basic MPI data types together with a displacement for each of the data types.*
- Trapezoidal Rule example:

| Variable | Address |
|---|---|
| a | 24 |
| b | 40 |
| n | 48 |

$$\{(\texttt{MPI_DOUBLE}, 0), (\texttt{MPI_DOUBLE}, 16), (\texttt{MPI_INT}, 24)\}$$

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Take timing: MPI_Wtime and gettimeofday

## MPI_Wtime

Returns an elapsed time on the calling processor

### Synopsis

```
double MPI_Wtime( void )
```

### Return value

Time in seconds since an arbitrary time in the past.

### Notes

This is intended to be a high-resolution, elapsed (or wall) clock. See `MPI_WTICK` to determine the resolution of `MPI_WTIME`. If the attribute `MPI_WTIME_IS_GLOBAL` is defined and true, then the value is synchronized across all processes in `MPI_COMM_WORLD`.

### Notes for Fortran

This is a function, declared as `DOUBLE PRECISION MPI_WTIME()` in Fortran.

See Also also: MPI_Wtick, MPI_Comm_get_attr, MPI_Attr_get

**Location:**src/mpi/timer/wtime.c

https://www.mpich.org/static/docs/v3.1/www3/MPI_Wtime.html

```
GETTIMEOFDAY(2)           FreeBSD System Calls Manual          GETTIMEOFDAY(2)

NAME
     gettimeofday, settimeofday -- get/set date and time

LIBRARY
     Standard C Library (libc, -lc)

SYNOPSIS
     #include <sys/time.h>

     int
     gettimeofday(struct timeval *tp, struct timezone *tzp);

     int
     settimeofday(const struct timeval *tp, const struct timezone *tzp);

DESCRIPTION
     The system's notion of the current Greenwich time and the current time
     zone is obtained with the gettimeofday() system call, and set with the
     settimeofday() system call.  The time is expressed in seconds and mi-
     croseconds since midnight (0 hour), January 1, 1970.  The resolution of
     the system clock is hardware dependent, and the time may be updated con-
     tinuously or in "ticks".  If tp or tzp is NULL, the associated time in-
     formation will not be returned or set.

     The structures pointed to by tp and tzp are defined in <sys/time.h> as:

     struct timeval {
             time_t          tv_sec;         /* seconds */
             suseconds_t     tv_usec;        /* and microseconds */
     };

     struct timezone {
             int     tz_minuteswest; /* minutes west of Greenwich */
             int     tz_dsttime;     /* type of dst correction */
     };

     The timezone structure indicates the local time zone (measured in minutes
     of time westward from Greenwich), and a flag that, if nonzero, indicates
     that Daylight Saving time applies locally during the appropriate part of
     the year.
```

https://www.freebsd.org/cgi/man.cgi?query=gettimeofday&sektion=2&n=1


Copyright © 2010, Elsevier Inc. All rights Reserved

# Run over multiple configurations

```
#PBS -l select=2:ncpus=1:mem=2gb -l place=pack
#PBS -l select=2:ncpus=1:mem=2gb -l place=scatter
#PBS -l select=2:ncpus=1:mem=2gb -l place=pack:excl
#PBS -l select=2:ncpus=1:mem=2gb -l place=scatter:excl
```

Important for your project

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Collective and data distribution

- MPI_Broadcast
- MPI_Reduce
- MPI_AllReduce

- MPI_Scatter
- MPI_Gather
- MPI_AllGather


Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Barrier

- Ensures that no process will return from calling it until every process in the communicator has started calling it.

```
int MPI_Barrier(MPI_Comm    comm    /* in */);
```

- Why are we talking about MPI_Barrier here?



MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# High Performance Computing for Data Science

Performance and MPI send/receive modes
Lecture 13 - 29/10/2026

## Performance

Prof. Sandro Fiore
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

# Matrix-vector multiplication

$A = (a_{ij})$ is an $m \times n$ matrix

$\mathbf{x}$ is a vector with $n$ components

$\mathbf{y} = A\mathbf{x}$ is a vector with $m$ components

$$y_i = a_{i0}x_0 + a_{i1}x_1 + a_{i2}x_2 + \cdots a_{i,n-1}x_{n-1}$$

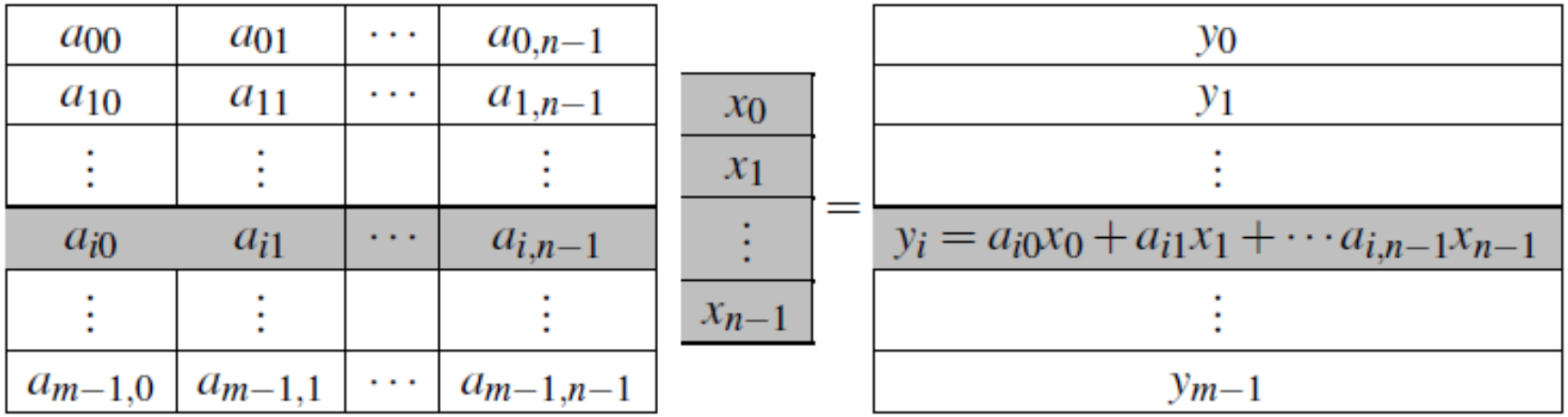
*i-th component of y*

*Dot product of the ith row of A with x.*

MK MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Matrix-vector multiplication




MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Run-times of serial and parallel matrix-vector multiplication

| comm_sz | Order of Matrix | | | | |
|---|---|---|---|---|---|
| | 1024 | 2048 | 4096 | 8192 | 16,384 |
| 1 | 4.1 | 16.0 | 64.0 | 270 | 1100 |
| 2 | 2.3 | 8.5 | 33.0 | 140 | 560 |
| 4 | 2.0 | 5.1 | 18.0 | 70 | 280 |
| 8 | 1.7 | 3.3 | 9.8 | 36 | 140 |
| 16 | 1.7 | 2.6 | 5.9 | 19 | 71 |

*(Seconds)*

- What can we argue from this table in terms of performance?

MK MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Speedup



- Number of cores = p
- Serial run-time = $T_{serial}$
- Parallel run-time = $T_{parallel}$

linear speedup

$$T_{parallel} = T_{serial} / p$$


Copyright © 2010, Elsevier Inc. All rights Reserved

# Speedup of a parallel program

$$S = \frac{T_{serial}}{T_{parallel}}$$

- What can we say about S in case of linear speedup?

$$T_{parallel} = T_{serial} / p$$

MK MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Efficiency of a parallel program

$$E = \frac{S}{p} = \frac{\left(\frac{T_{serial}}{T_{parallel}}\right)}{p} = \frac{T_{serial}}{p \cdot T_{parallel}}$$

- What can we say about E in case of linear speedup?

$$T_{parallel} = T_{serial} / p$$

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Speedups and efficiencies of a parallel program

| $p$ | 1 | 2 | 4 | 8 | 16 |
|---|---|---|---|---|---|
| $S$ | 1.0 | 1.9 | 3.6 | 6.5 | 10.8 |
| $E = S/p$ | 1.0 | 0.95 | 0.90 | 0.81 | 0.68 |

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Effect of overhead

We defined linear speedup as

$$T_{parallel} = T_{serial} / p$$

In general, we should also account for an overhead contribution

$$T_{parallel} = T_{serial} / p + T_{overhead}$$

MK MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Amdahl's Law

- *Unless virtually all of a serial program is parallelized, the possible speedup is going to be very limited — regardless of the number of cores available*




Copyright © 2010, Elsevier Inc. All rights Reserved

# Example

- We can parallelize 90% of a serial program.
- Parallelization is “perfect” regardless of the number of cores *p* we use.
- $T_{serial} = 20$ seconds
- Runtime of parallelizable part is

$$0.9 \times T_{serial} / p = 18 / p$$


Copyright © 2010, Elsevier Inc. All rights Reserved

# Example (cont.)

- Runtime of "unparallelizable" part is

$$0.1 \times T_{serial} = 2$$

- Overall parallel run-time is

$$T_{parallel} = 0.9 \times T_{serial} / p + 0.1 \times T_{serial} = 18 / p + 2$$

$$T_{parallel} = T_{serial} / p + T_{overhead}$$


Copyright © 2010, Elsevier Inc. All rights Reserved

# Example (cont.)

- Speed up

$$S = \frac{T_{serial}}{0.9 \times T_{serial} / p + 0.1 \times T_{serial}} = \frac{20}{18 / p + 2}$$

- What if I run a parallel test with p=1000? or bigger?


Copyright © 2010, Elsevier Inc. All rights Reserved

# Speedups and efficiencies of a parallel program… we started from here

| $p$ | 1 | 2 | 4 | 8 | 16 |
|---|---|---|---|---|---|
| $S$ | 1.0 | 1.9 | 3.6 | 6.5 | 10.8 |
| $E = S/p$ | 1.0 | 0.95 | 0.90 | 0.81 | 0.68 |

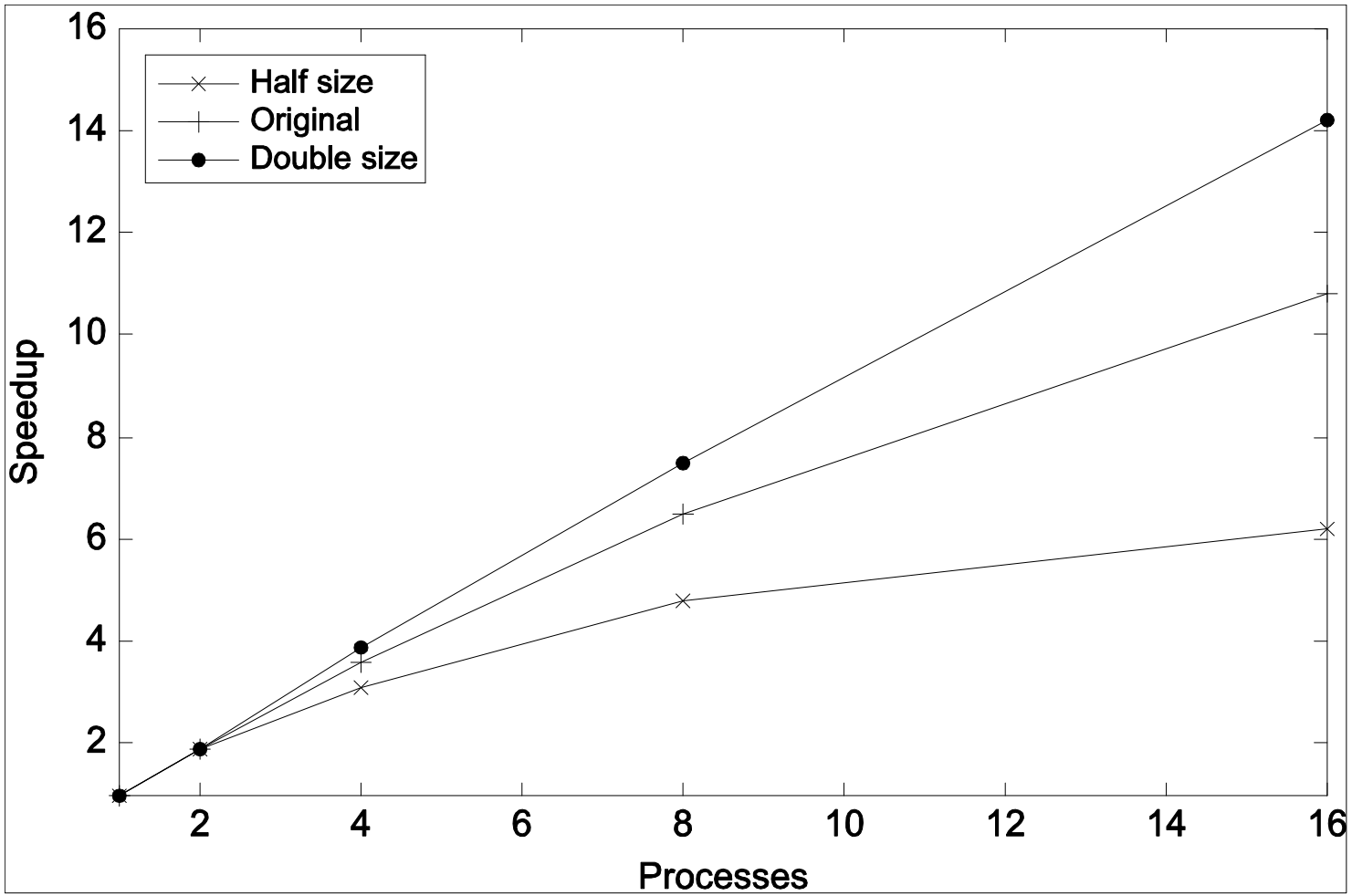
MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Speedups and efficiencies of parallel program on different problem sizes

| | $p$ | 1 | 2 | 4 | 8 | 16 |
|---|---|---|---|---|---|---|
| Half | $S$ | 1.0 | 1.9 | 3.1 | 4.8 | 6.2 |
| | $E$ | 1.0 | 0.95 | 0.78 | 0.60 | 0.39 |
| Original | $S$ | 1.0 | 1.9 | 3.6 | 6.5 | 10.8 |
| | $E$ | 1.0 | 0.95 | 0.90 | 0.81 | 0.68 |
| Double | $S$ | 1.0 | 1.9 | 3.9 | 7.5 | 14.2 |
| | $E$ | 1.0 | 0.95 | 0.98 | 0.94 | 0.89 |

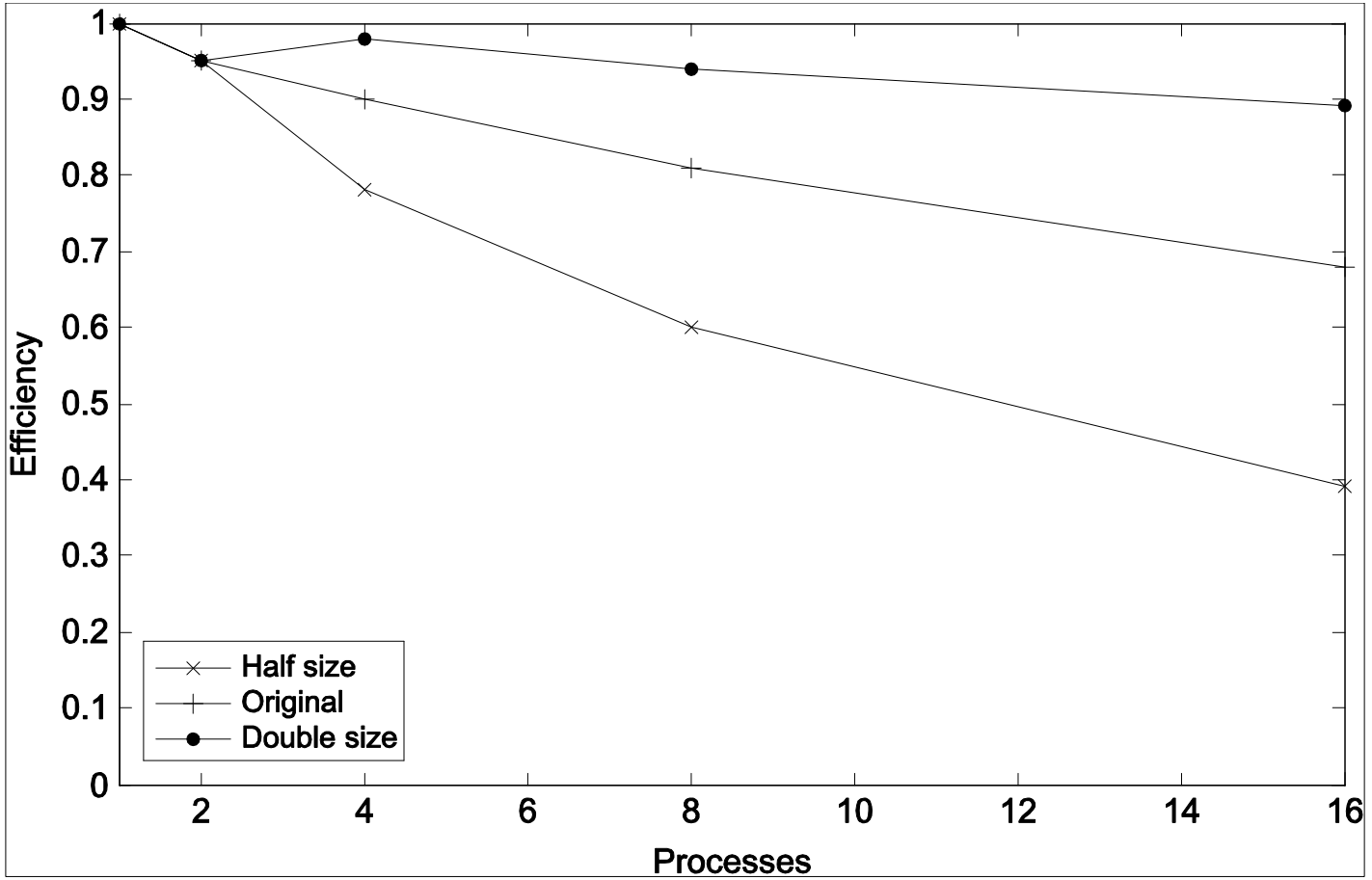
MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Speedup




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Efficiency




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Speedup & Efficiency

$$S(n,p) = \frac{T_{\text{serial}}(n)}{T_{\text{parallel}}(n,p)}$$

$$E(n,p) = \frac{S(n,p)}{p} = \frac{T_{\text{serial}}(n)}{p \times T_{\text{parallel}}(n,p)}$$

**The problem (n) size matters!**

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Run-times of serial and parallel matrix-vector multiplication

| comm_sz | Order of Matrix | | | | |
|---|---|---|---|---|---|
| | 1024 | 2048 | 4096 | 8192 | 16,384 |
| 1 | 4.1 | 16.0 | 64.0 | 270 | 1100 |
| 2 | 2.3 | 8.5 | 33.0 | 140 | 560 |
| 4 | 2.0 | 5.1 | 18.0 | 70 | 280 |
| 8 | 1.7 | 3.3 | 9.8 | 36 | 140 |
| 16 | 1.7 | 2.6 | 5.9 | 19 | 71 |

*(Seconds)*

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Speedups of Parallel Matrix-Vector Multiplication

| comm_sz | Order of Matrix | | | | |
|---|---|---|---|---|---|
| | 1024 | 2048 | 4096 | 8192 | 16,384 |
| 1 | 1.0 | 1.0 | 1.0 | 1.0 | 1.0 |
| 2 | 1.8 | 1.9 | 1.9 | 1.9 | 2.0 |
| 4 | 2.1 | 3.1 | 3.6 | 3.9 | 3.9 |
| 8 | 2.4 | 4.8 | 6.5 | 7.5 | 7.9 |
| 16 | 2.4 | 6.2 | 10.8 | 14.2 | 15.5 |

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Efficiencies of Parallel Matrix-Vector Multiplication

| comm_sz | Order of Matrix | | | | |
|---|---|---|---|---|---|
| | 1024 | 2048 | 4096 | 8192 | 16,384 |
| 1 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| 2 | 0.89 | 0.94 | 0.97 | 0.96 | 0.98 |
| 4 | 0.51 | 0.78 | 0.89 | 0.96 | 0.98 |
| 8 | 0.30 | 0.61 | 0.82 | 0.94 | 0.98 |
| 16 | 0.15 | 0.39 | 0.68 | 0.89 | 0.97 |

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Scalability

- A program is scalable if the problem size can be increased at a rate so that the efficiency doesn't decrease as the number of processes increase.




Copyright © 2010, Elsevier Inc. All rights Reserved

# Scalability

- Programs that can maintain a *constant efficiency* without increasing the problem size are sometimes said to be strongly scalable.

- Programs that can maintain a *constant efficiency* if the problem size increases at the same rate as the number of processes are sometimes said to be weakly scalable.


Copyright © 2010, Elsevier Inc. All rights Reserved

# What can we argue about scalability?

| comm_sz | Order of Matrix | | | | |
|---|---|---|---|---|---|
| | 1024 | 2048 | 4096 | 8192 | 16,384 |
| 1 | 1.00 | 1.00 | 1.00 | 1.00 | 1.00 |
| 2 | 0.89 | 0.94 | 0.97 | 0.96 | 0.98 |
| 4 | 0.51 | 0.78 | 0.89 | 0.96 | 0.98 |
| 8 | 0.30 | 0.61 | 0.82 | 0.94 | 0.98 |
| 16 | 0.15 | 0.39 | 0.68 | 0.89 | 0.97 |

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# High Performance Computing for Data Science

Performance and MPI send/receive modes
Lecture 13 - 29/10/2026

## MPI send and receive modes

Prof. Sandro Fiore
Department of Information Engineering and Computer Science (DISI)
University of Trento, 2026-2027

# Send and receive modes

### A quick overview of MPI's send modes

MPI has a number of different "send modes." These represent different choices of *buffering* (where is the data kept until it is received) and *synchronization* (when does a send complete). In the following, I use "send buffer" for the user-provided buffer to send.

MPI_Send
: MPI_Send will not return until you can use the send buffer. It may or may not block (it is allowed to buffer, either on the sender or receiver side, or to wait for the matching receive).

MPI_Bsend
: May buffer; returns immediately and you can use the send buffer. A late add-on to the MPI specification. Should be used only when absolutely necessary.

MPI_Ssend
: will not return until matching receive posted

MPI_Rsend
: May be used ONLY if matching receive already posted. User responsible for writing a correct program.

MPI_Isend
: Nonblocking send. But not necessarily asynchronous. You can NOT reuse the send buffer until either a successful, wait/test or you KNOW that the message has been received (see MPI_Request_free). Note also that while the **I** refers to *immediate*, there is no performance requirement on MPI_Isend. An immediate send must return to the user without requiring a matching receive at the destination. An implementation is free to send the data to the destination before returning, as long as the send call does not block waiting for a matching receive. Different strategies of when to send the data offer different performance advantages and disadvantages that will depend on the application.

MPI_Ibsend
: buffered nonblocking

MPI_Issend
: Synchronous nonblocking. Note that a Wait/Test will complete only when the matching receive is posted.

MPI_Irsend
: As with MPI_Rsend, but nonblocking.

Note that "nonblocking" refers ONLY to whether the data buffer is available for reuse after the call. No part of the MPI specification, for example, mandates concurent operation of data transfers and computation.

Some people have expressed concern about not having a single "perfect" send routine. But note that in general you can't write code in Fortran that will run at optimum speed on both Vector and RICS/Cache machines without picking different code for the different architectures. MPI at least lets you express the different algorithms, just like C or Fortran.

### Recommendations

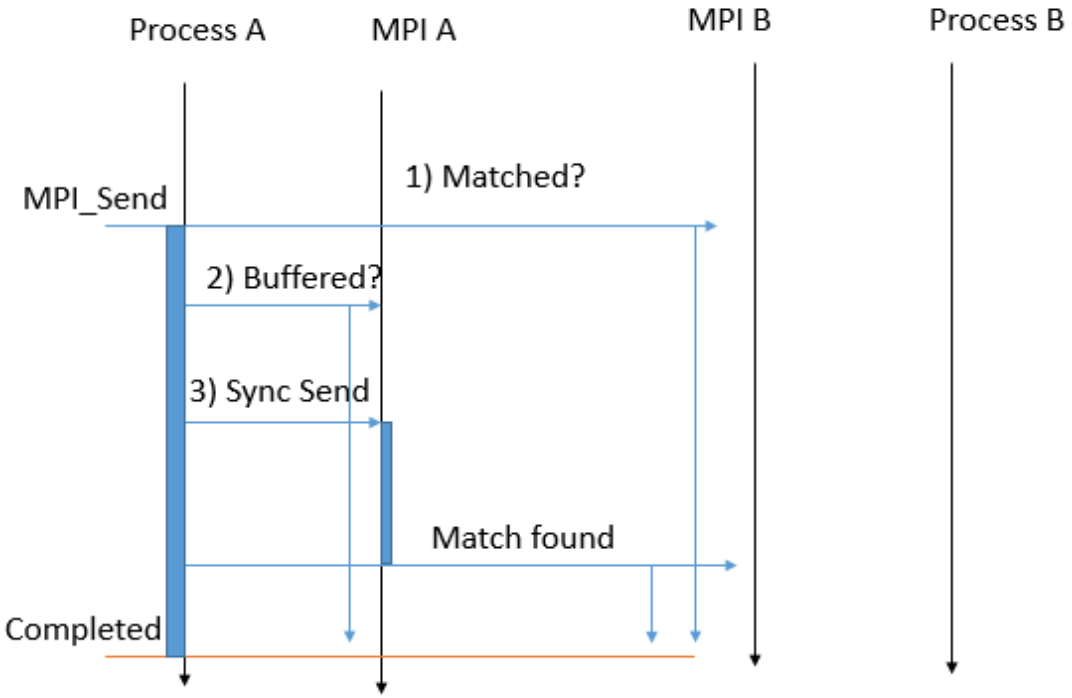
The best performance is likely if you can write your program so that you could use just MPI_Ssend; in that case, an MPI implementation can completely avoid buffering data. Use MPI_Send instead; this allows the MPI implementation the maximum flexibility in choosing how to deliver your data. (Unfortunately, one vendor has chosen to have MPI_Send emphasize buffering over performance; on that system, MPI_Ssend may perform better.) If nonblocking routines are necessary, then try to use MPI_Isend or MPI_Irecv. Use MPI_Bsend only when it is too inconvienent to use MPI_Isend. The remaining routines, MPI_Rsend, MPI_Issend, etc., are rarely used but may be of value in writing system-dependent message-passing code entirely within MPI.

Reference: https://www.mcs.anl.gov/research/projects/mpi/sendmode.html


Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Send (Standard mode)

- This is the **standard** mode
- When it is called,
  - (1) the message can be directly passed to the receive buffer
  - (2) the data is buffered (in temporary memory in the MPI implementation) or
  - (3) the function waits for a receiving process to appear.
- Therefore, it can return quickly (1)(2) or block the process for a while (3).




- Note: MPI decides which scenario is the best in terms of performance, memory, and so on. In any case, the data can be safely modified after the function returns.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

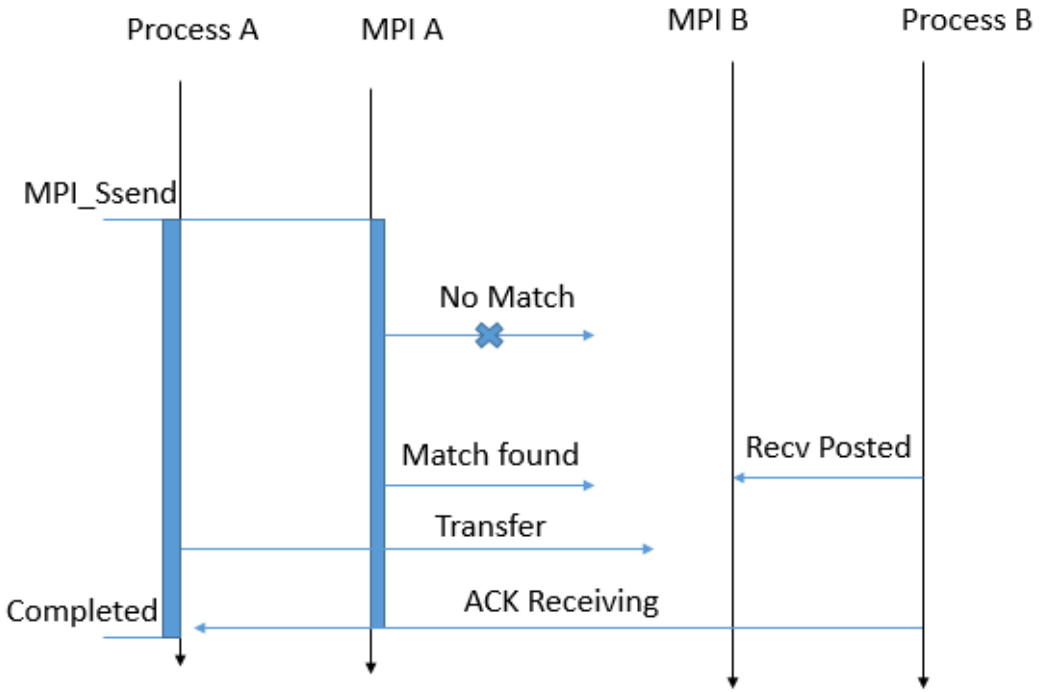
# MPI_Ssend, MPI_Bsend, MPI_Rsend (blocking functions)

- MPI_Ssend()
  - **synchronization** of source and destination
  - the behavior is predictable and safe
  - recommend for debugging purpose
- MPI_Bsend()
  - only does copy message to **buffer**
  - completes immediately
  - predictable behavior and no synchronization
  - user must allocate extra buffer space by MPI_Buffer_attach()
- MPI_Rsend()
  - completes immediately
  - will succeed only if a matching receive is already posted
  - if receiving process is not **ready**, action is undefined.
  - may improve performance


Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Ssend (S = Synchronous)

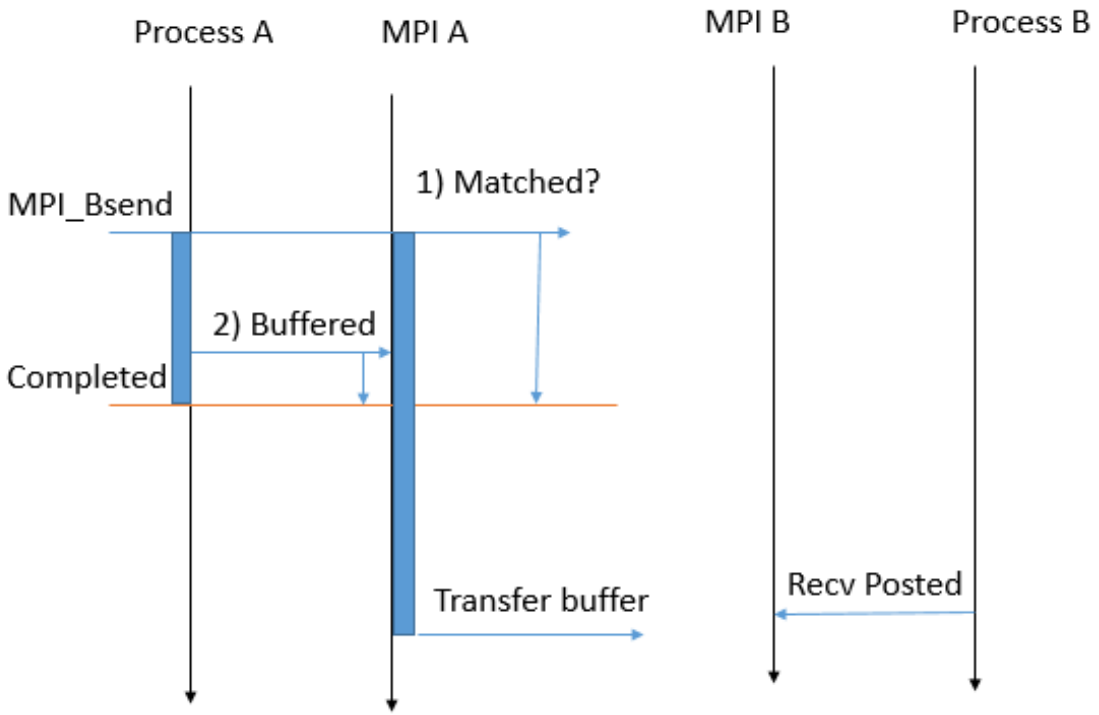
- It is the **synchronized** blocking function. *When this function returns, the destination has started receiving the message*. The moment the destination starts receiving, it signals an *ACK* to the source.




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Bsend (B = Buffered)

- It is the **local blocking send**. The programmer defines a local buffer when this function is called
- If there is not a matching receive available, the process is blocked until the message is copied into the buffer.
- Therefore, the programmer can immediately modify the source data after the function returns.




- Note, when the function returns the message is probably not sent yet, it will happen concurrently in the background of the process when a matched receive is found.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Rsend (R = Ready)

- It is a blocking function the same as MPI_Send, but it expects a ready destination to receive the message.
- This can increase the MPI performance if the programmer is sure there is a receive function waiting for this.
- If no receive posted before, it is erroneous.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Non blocking communication (I)

## MPI_Isend, MPI_Issend, MPI_Ibsend, MPI_Irsend

- Advantages:
  - allows the separation between the **initialization** of the communication and the **completion**
  - can avoid deadlock
  - can reduce latency by posting receive calls early
- Disadvantages:
  - complex to develop, maintain and debug code

```
int MPI_Isend(void* message /* in */,
    int               count /* in */,
    MPI_Datatype      datatype /* in */,
    int               dest /* in */,
    int               tag /* in */,
    MPI_Comm          comm /* in */,
    MPI_Request*      request /* out */)

int MPI_Irecv(void* message /* out */,
    int               count /* in */,
    MPI_Datatype      datatype /* in */,
    int               source /* in */,
    int               tag /* in */,
    MPI_Comm          comm /* in */,
    MPI_Request*      request /* out */)
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Non blocking communication (II)

## MPI_Isend, MPI_Issend, MPI_Ibsend, MPI_Irsend

- Non-blocking operation requires a minimum of two function calls: a call to start the operation and a call to complete the operation.
  - The "***request***" is used to query the status of the communicator or to wait for its completion.
  - The user must NOT overwrite the send buffer until the send (data transfer) is complete.
  - The user can not use the receiving buffer before the receive is complete
  - **MPI_Wait()** returns when the operation is complete, and the status is updated for a receive.
  - **MPI_Test()** returns immediately, with "flag" = true if posted operation corresponding to the "request" handle is complete.

```
int MPI_Wait(MPI_Request*     request /* in-out */,
             MPI_Status*      status /* out */)

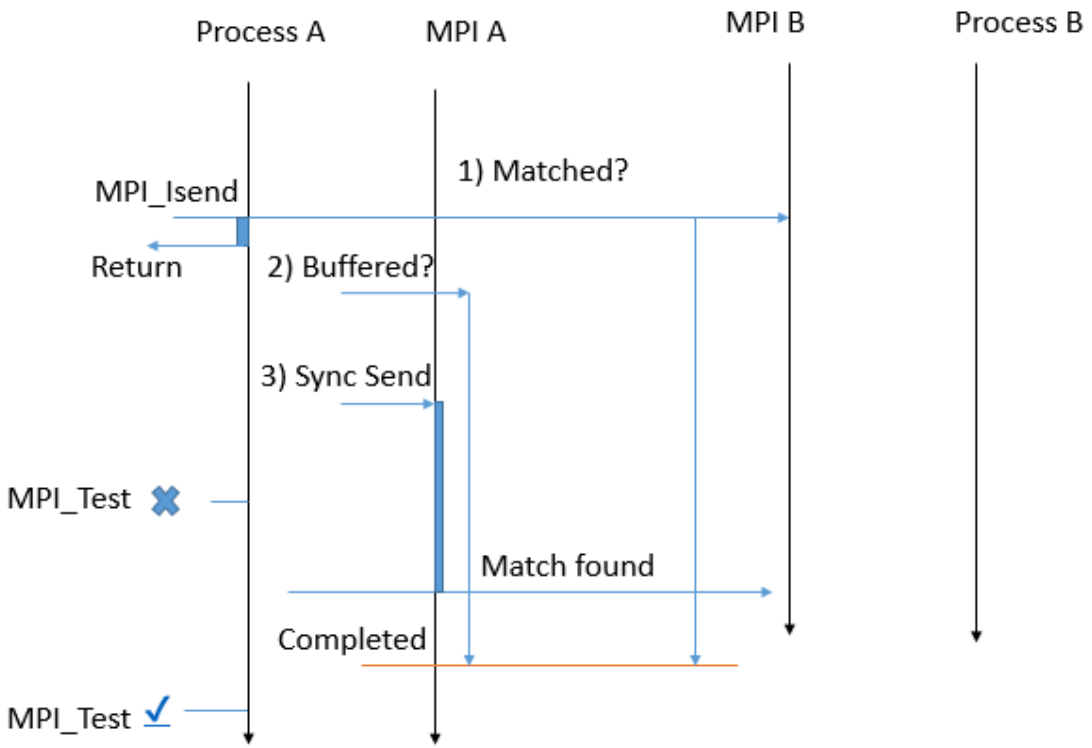
int MPI_Test(MPI_Request*     request /* out */,
             int*             flag /* out*/,
             MPI_Status*      status /* out */)
```

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Isend

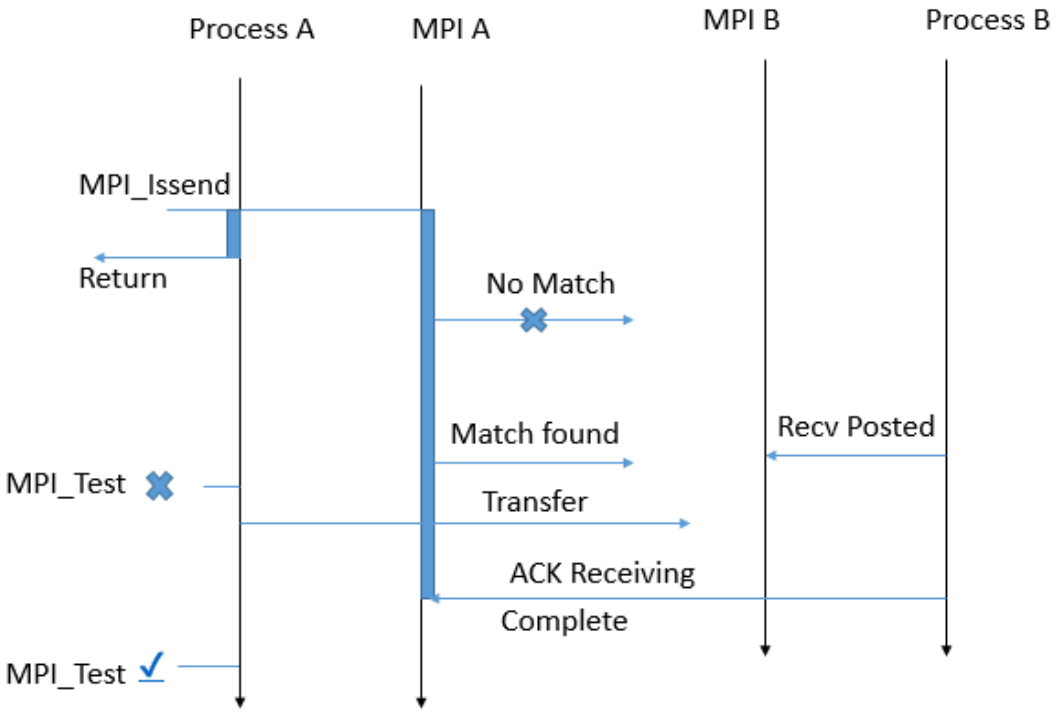
- **It is the non-blocking version of MPI_Send**
- When this function is called the function returns immediately but runs MPI_Send actions in the background of the process
- Therefore, After the function returns, the data must not be modified unless MPI_Test and MPI _Wait confirm MPI_Isend i s completed
- After the completion, the data is reusable because it either is buffered in MPI or sent to the destination




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Issend (Synchronized)

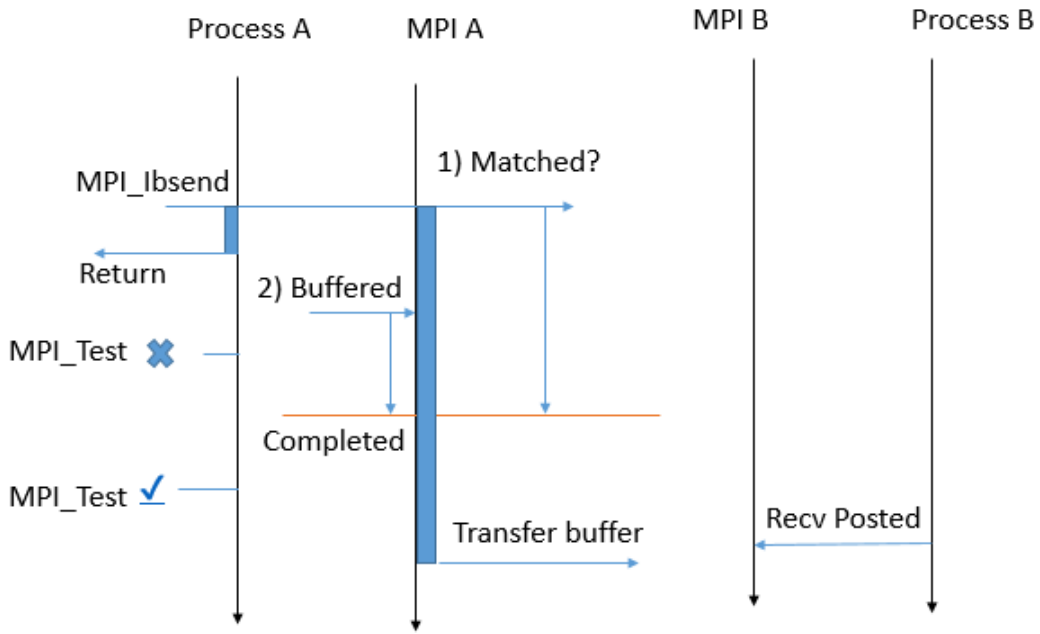
- **It is the non-blocking version of MPI_Ssend.**
- It returns immediately, but runs MPI_Ssend actions in the background.
- MPI_Test or MPI_Wait must be used to assess if the function is completed in the background.
- At that point, not only the message has been sent but also the destination has started to receive the message




MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# MPI_Ibsend and MPI_Irsend

- **MP_Ibsend (*Buffered*):**
  - This is the local non-blocking of Bsend.
  - It blocks for neither copying the message to the buffer nor sending the message.
  - After the test positive or wait, we can modify the source data because, if it is not sent, it is locally copied to the allocated buffer.




- **MPI_Irsend (*Ready*):**
  - The same as MPI_Rsend, but non-blocking.

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Summarizing

| Send Modes | MPI function | Completion Condition |
|---|---|---|
| Synchronous send | MPI_Ssend() | A send will not complete until a matching receive has been posted and the matching receive has begun reception of the data. |
| | MPI_Issend() | |
| Buffered send (It has additional associated functions. The send operation is **local**.) | MPI_Bsend() | Bsend() always completes (unless an error occurs) Completion is irrespective of the receiver. |
| | MPI_Ibsend() | |
| **Standard send | MPI_Send() | message sent (no guarantee that the receive has started). It is up to MPI to decide what to do. |
| | MPI_Isend() | |
| Ready send | MPI_Rsend() | may be used only when the a matching receive has already been posted |
| | MPI_Irsend() | |

MORGAN KAUFMANN
Copyright © 2010, Elsevier Inc. All rights Reserved

# Some more general & useful tips

- When there is a **deadlock**, non-blocking communication can help.
- When there is a **race condition**, blocking communication can help.
- When there is a **computationally expensive** task, a non-blocking communication, posted before the task, may improve the performance.
- MPI_Send and MPI_Isend, are **top** in the priority list as MPI decides what is best.
- MPI_Ssend and MPI_Issend, when the sender needs to know **when** the message is received and to avoid local buffering. MPI_Ssend can be useful for debugging.
- MPI_Bsend, MPI_Ibsend, MPI_Rsend, MPI_Irsend are for **fine-tuning** the performance.

- Credits:
  - https://iamsorush.com/posts/mpi-send-types/


Copyright © 2010, Elsevier Inc. All rights Reserved

# Hands-on: Performance benchmark

**Matrix Vector multiplication**

$$\begin{bmatrix} a_{00} & a_{01} & \cdots & a_{0,n-1} \\ a_{10} & a_{11} & \cdots & a_{1,n-1} \\ \vdots & \vdots & & \vdots \\ a_{i0} & a_{i1} & \cdots & a_{i,n-1} \\ \vdots & \vdots & & \vdots \\ a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1} \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix} = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1} \\ \vdots \\ y_{m-1} \end{bmatrix}$$

MORGAN KAUFMANN

Copyright © 2010, Elsevier Inc. All rights Reserved

# Feedback form

- Throughout the entire HPC4DS course there will be a form available for your feedback
  - Please provide any comment about:
    - Pros
    - Cons
    - Aspects that were not clear enough during the class
    - Any other feedback you think can be relevant for the course
    - …
- Information are gathered in an anonymous way

https://docs.google.com/forms/d/e/1FAIpQLScSVpNPfMP8poNNz-nLl9xlKlHkeBM2x_JgWyA0qQu6mOBl5A/viewform